

Predicting Season-Level NFL Team Success with Multi-Source Machine Learning

Seth Phillips¹, Brittany Fetterman¹, Ethan Stanks¹

Abstract

NFL analytics research often studies single games, single plays, or single players. This study asks a broader question. It tests whether season-level NFL team wins can be predicted from a combined team-season dataset. Public NFL data from 2015 to 2024 were collected with `nfl_data.py` and merged into one table with 320 team-season observations. The final dataset included play-by-play totals, roster information, injury reports, draft data, schedule strength, and lagged win features. Seven regression models were evaluated with a chronological split. Seasons 2015 to 2021 were used for training, and seasons 2022 to 2024 were used for testing. The LSTM model performed best, with an MSE of 1.279, RMSE of 1.131, MAE of 0.887, and R^2 of 0.907. GRU and Ridge Regression followed closely, and both reached an R^2 of 0.905. In the tree models, the most important variables were total yards, offensive win probability added, first downs, defensive EPA per play, and rushing attempts. These results suggest that season-level team wins can be modeled well in this dataset when several team-level data sources are merged into one framework.

Keywords: NFL, machine learning, team success, season wins, LSTM, GRU, sports analytics

1 Introduction

NFL team success is often measured by wins, but wins do not come from one part of the game alone. Over a full season, successful teams usually need steady offense, solid defense, roster depth, limited injury loss, and a manageable schedule. Much of NFL analytics focuses on smaller parts of the game. Many studies predict one game, one play, one penalty, or one player's career outcome. Those studies are useful, but they do not fully explain why a team succeeds over a full season. The research question for this study is: *Can a multi-source, season-level machine learning dataset predict the number of wins an NFL team earns in a season?*

To answer that question, this project builds a team-season dataset for the 2015 to 2024 NFL seasons. It combines play-by-play data, roster data, injury data, draft data,

schedule data, and historical wins. The target variable is total wins in a season. The models predict a continuous number of wins, not a binary win-loss label. The main contribution of this project is its focus on season-level team success. The paper does not stop at game-level prediction. It also does not study only one part of football, such as passing or combine data. Instead, it merges several forms of team context into one modeling pipeline and compares simple models with more complex models.

2 Literature Review

Sports analytics uses data to explain and predict performance. In NFL research, many studies focus on single-game outcomes. Gifford and Bayrak used decision tree and logistic regression models to forecast NFL game outcomes. Their study used team statistics as predictors and a binary win-loss outcome as the target. They found that turnovers were among the strongest predictors of winning a game. This supports the idea that team statistics can explain football results, but the target was still a single-game result [Gifford, Matt and Bayrak, Tuncay \(2023\)](#).

Juuri also studied NFL game prediction with machine learning. That work focused on predicting the results of NFL games and used game statistics as the main input. It showed that model accuracy depends on feature quality and on how historical football data are handled. This line of work is close to the current project, but the prediction level is still the game [Juuri, Sebastian \(2023\)](#).

Weirich, Kim, Yoon, and Jeong moved closer to season-level success by comparing Pythagorean expectation, Random Forest, and neural network models for NFL winning percentage. Their machine learning models performed better than the traditional Pythagorean formula. Their neural network model reached an R^2 of 0.891 for winning percentage. This is one of the closest studies to the current project because it studies winning percentage over a full season [Weirich, Caroline et al. \(2025\)](#).

Other NFL studies use detailed play-level data. Da Silva and Moral used tracking data to estimate frame-by-frame pass completion probability. Skoki, Lerga, and Stajduhar used GPS tracking data to predict defensive pass interference. These studies show the value of detailed football data, but they stay at the level of single plays [da Silva, Gustavo Pompeu and de Andrade Moral, Rafael \(2021\)](#); [Skoki, Arian et al. \(2022\)](#).

Win probability is another common football analytics topic. Brill, Yurko, and Wyner studied the difficulty of estimating win probability from football game states. They found that play-by-play observations are noisy and highly related within games. This can inflate bias and variance and make the sample size look larger than it really is. This matters for the current study because a team-season row still contains many related events from the same season [Brill, Ryan S. et al. \(2025\)](#).

Some studies focus on player-level results. Szekely, Sinnott, Halow, and Ryan studied whether NFL Scouting Combine data could predict career success. They found that combine data could predict whether a player would play an NFL snap with about 83 percent accuracy, but it did not predict long-term snap counts well [Szekely, Brian et al. \(2023\)](#). Mahoney and Paniak studied daily fantasy football lineup creation with machine learning and linear programming. Their work predicted player fantasy points

and used optimization to build lineups, but it did not directly model how real NFL teams win over a season [Mahoney, Joseph M. and Paniak, Tomasz B. \(2023\)](#).

Obradovic and Keco reviewed sports result prediction with machine learning and argued that domain knowledge matters when choosing features for sports models. That supports the design of the current project, which uses football-based features such as yards, turnovers, injuries, roster size, draft value, and strength of schedule [Obradović, Alden and Kečo, Dino \(2024\)](#).

Taken together, prior research shows that machine learning can help predict NFL outcomes. It also shows a clear gap. Many studies predict games, plays, penalties, fantasy points, or player careers. Fewer studies combine several team-level data sources to predict season wins. This paper is built around that gap.

3 Data and Methodology

3.1 Data Sources

The dataset covers the 2015 to 2024 NFL seasons. Each observation represents one team in one season. The final dataset has 320 team-season observations and 39 columns after feature engineering. The raw sources were collected with `nfl_data.py`. Table 1 lists the main sources and shows how each source was used in the study.

Table 1 Main data sources used in the project

Source	Use in the study
Seasonal player data	Player and team production measures
Play-by-play data	Offensive and defensive team totals by season
Seasonal rosters	Roster size, average experience, and quarterback count
Injury reports	Injury report counts and out or injured reserve counts
Draft picks	Number of picks, draft value proxy, and draft round measures
Schedules	Actual wins and strength of schedule
Win totals data	Collected as market context, but final target wins were computed from game results

The play-by-play data were large, so each season was loaded one at a time. The data were grouped by season and team before they were joined to the master table. This reduced the data to one row per team-season.

3.2 Target Variable

The target variable is *actual_wins*. It was computed from schedule results. For each completed game, a win was assigned to the home team or away team based on the final score. Wins were then summed by team and season. The target variable ranged from 0 to 18 wins in the final dataset. In this paper, the maximum is 18 because postseason games were included when schedule records were aggregated.

3.3 Preprocessing

Each source was reduced to the same unit of analysis, one team in one season. After the sources were merged, missing values were checked. Draft variables had the highest missing rate at 23.8 percent. Play-by-play variables had 2.5 percent missing. Roster variables had 1.6 percent missing. Numeric missing values were filled with the median. Rows without the target variable were dropped. After imputation, the dataset had no missing values.

Numerical features were scaled with `StandardScaler`. The scaler was fit on the training set only and then applied to the test set. Team names were label encoded into a numeric team identifier.

3.4 Feature Engineering

Several derived features were created from the raw aggregates. Completion percentage was computed as completions divided by pass attempts. Yards per play was computed as total yards divided by the sum of pass attempts and rush attempts. Pass-run ratio was computed as pass attempts divided by rush attempts. Turnovers were defined as interceptions thrown plus fumbles. An injury burden score was computed as total injury reports divided by roster size. A draft quality score was computed as total draft value divided by total picks. Two lagged success features were also added, prior season wins and a two-year average of prior wins. These lagged variables gave the model a way to use past team performance.

The final feature set had 36 predictors. These features included offensive production, defensive production, roster construction, injury burden, draft information, schedule strength, lagged wins, and team ID.

3.5 Modeling Methods

The models were trained with a chronological split. Seasons 2015 to 2021 were used for training, and seasons 2022 to 2024 were used for testing. After rows with missing lagged win features were removed, the analytic sample used for modeling contained 251 team-season observations. The training set had 155 observations, and the test set had 96 observations. In total, 69 rows were excluded at this stage. Most of those rows were early team-history rows where `prev_season_wins` or `wins_2yr_avg` could not yet be computed. Some extra row loss also came from team label changes across seasons, because the lag features were grouped by team label in the notebook. Seven regression models were tested:

- Linear Regression was used as the baseline model. It fits a direct weighted relationship between the features and season wins.
- Ridge Regression was used as a regularized baseline. It adds an L2 penalty, which helps when predictors are related to each other.
- Random Forest Regression was used as an ensemble tree model. It builds many decision trees and averages their predictions.
- XGBoost Regression was used as a boosted tree model. It builds trees one after another, with each new tree trying to correct prior errors.

- A multi-layer perceptron, or MLP, was used as a feedforward neural network. It was built in PyTorch with hidden layers, batch normalization, ReLU activation, dropout, and one output for predicted wins.
- GRU and LSTM neural networks were also tested. The input was reshaped into a three-dimensional format with one time step. The time signal came mainly from lagged win features, not from long play sequences. Optuna tuned hidden size, number of layers, dropout, learning rate, and batch size for both models. Each search used 50 trials.

The evaluation metrics were mean squared error, root mean squared error, mean absolute error, and R^2 . These metrics were used for all models.

4 Results and Analysis

4.1 Descriptive Statistics

Table 2 shows summary statistics for key variables in the final dataset. Average team wins were 8.544. The standard deviation was 3.623 wins. The average strength of schedule was close to 0.500, which is expected because every team plays against other teams in the same league.

Table 2 Summary statistics for selected variables

Variable	Mean	Std. Dev.	Min	Max
Actual wins	8.544	3.623	0.000	18.000
Offensive EPA per play	-0.012	0.067	-0.212	0.174
Defensive EPA per play	-0.010	0.050	-0.178	0.121
Total injury reports	171.000	44.391	62.000	318.000
Strength of schedule	0.498	0.028	0.414	0.581
Total draft value	0.183	0.188	0.026	1.351
Average experience	3.085	0.514	1.835	4.596
Prior season wins	8.526	3.560	0.000	17.000

The descriptive results show that the dataset contains teams across a wide range of success. Some teams had very poor seasons, while other teams had very strong seasons. This range is useful for training a regression model because the target is not restricted to only average teams.

4.2 Model Comparison

Table 3 shows the final test results for all seven models. The LSTM model performed best by a small margin. It reached an RMSE of 1.131 wins and an R^2 of 0.907. GRU and Ridge Regression followed closely, and both reached an R^2 of 0.905. Linear Regression also performed well, with an R^2 of 0.900.

These results support the research question. The models predicted season-level team wins with strong accuracy. The best model missed by less than one win on average when measured by MAE.

Table 3 Test-set performance for all seven models. Lower MSE, RMSE, and MAE are better. Higher R^2 is better.

Model	MSE	RMSE	MAE	R^2
LSTM Optuna tuned	1.279	1.131	0.887	0.907
GRU Optuna tuned	1.310	1.144	0.910	0.905
Ridge Regression	1.298	1.139	0.894	0.905
Linear Regression	1.373	1.172	0.911	0.900
XGBoost	2.123	1.457	1.132	0.845
Random Forest	2.222	1.491	1.164	0.838
MLP Neural Network	3.070	1.752	1.290	0.776

The gap between the top models was small. The LSTM beat Ridge Regression by only 0.008 in RMSE and 0.002 in R^2 . This means the deep model had the best score, but the simpler regularized linear model was almost the same. In practical terms, the feature set carried most of the predictive power.

The MLP performed worse than the other neural models. This may be because the dataset was small for a deep learning model. The training set had only 155 rows after lagged features were used. A larger neural network can fit noise when the sample size is small.

The tree models performed lower than the linear and recurrent models. Random Forest reached an R^2 of 0.838 and XGBoost reached an R^2 of 0.845. These are still useful scores, but they did not match the top models. This may mean the relationship between the engineered features and wins was mostly captured by linear effects.

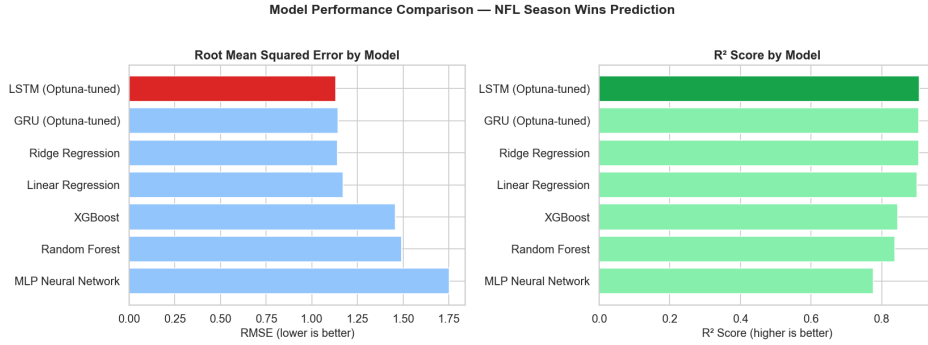


Fig. 1 Comparison of model performance on the test set. Lower RMSE and higher R^2 indicate better performance.

4.3 Feature Importance

Random Forest feature importance was used to inspect which variables mattered most in that model. Table 4 shows the top ten features. Total yards had the largest importance value at 0.368. Offensive WPA was second at 0.162. First downs, defensive EPA per play, and rushing attempts were also high.

Table 4 Top ten Random Forest feature importance values

Feature	Importance
Total yards	0.368
Offensive WPA	0.162
First downs	0.098
Defensive EPA per play	0.093
Rush attempts	0.074
Offensive touchdowns	0.045
Defensive interceptions	0.021
Strength of schedule	0.020
Sacks	0.017
Offensive EPA per play	0.010

These values describe the Random Forest model only. They do not show causal effects. They show which features the model used most when it split the data. Within that framework, team production mattered most. Total yards, first downs, and touchdowns show that the offense moved the ball and finished drives. Offensive WPA also mattered because it reflects how plays changed a team's chance of winning. Defensive EPA per play was important because it captures how much value a defense allowed or prevented on an average play.

The injury and draft features were included in the dataset, but they did not appear among the top Random Forest features. This does not mean they have no value. It means that, in this trained model, same-season performance features explained more variation in wins.

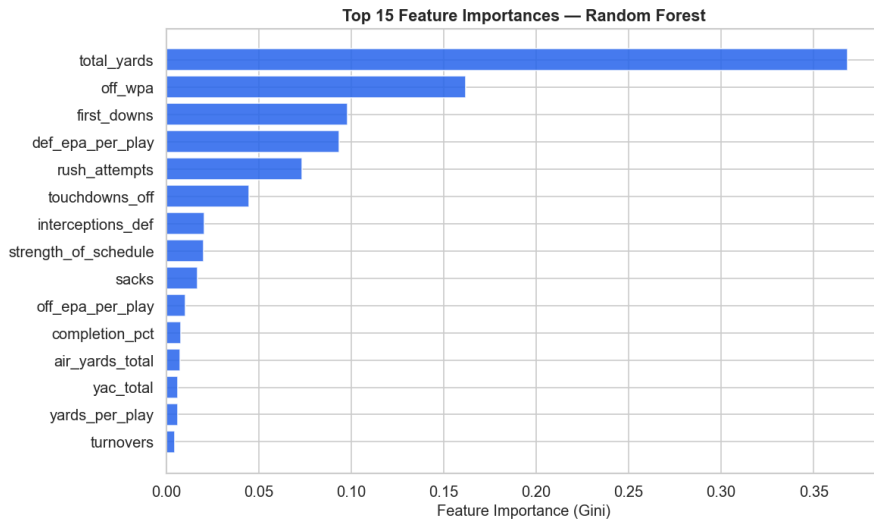


Fig. 2 Top Random Forest feature importance values. Larger values indicate greater importance within that model.

4.4 Actual Versus Predicted Wins

Figure 3 shows actual versus predicted wins for the XGBoost model. This plot was used as a visual diagnostic for a tree-based model. Points closer to the diagonal line indicate smaller prediction error. XGBoost had an RMSE of 1.457 and an R^2 of 0.845. The plot shows that the model followed the general pattern in team wins, but it still trailed the best models.

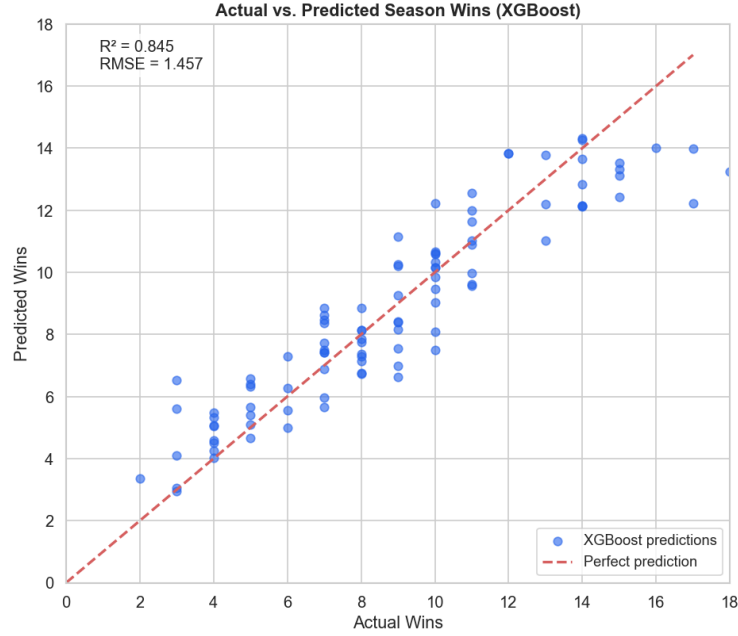


Fig. 3 Actual versus predicted season wins for XGBoost on the test set. Points closer to the diagonal line indicate smaller prediction error.

4.5 Connection to Prior Research

The results fit with the literature in several ways. Gifford and Bayrak found that team statistics can predict NFL game outcomes with decision trees and logistic regression [Gifford, Matt and Bayrak, Tuncay \(2023\)](#). This project found that aggregated team statistics can also predict season-level wins. Weirich et al. found that neural networks and Random Forest models can outperform traditional formulas for winning percentage [Weirich, Caroline et al. \(2025\)](#). This project also found that a tuned neural network had the highest score, but the margin over Ridge Regression was small.

The results also support a caution from Brill, Yurko, and Wyner. Football data can have dependence and uncertainty because many observations are connected [Brill, Ryan S. et al. \(2025\)](#). The current study used a chronological split to reduce direct time leakage, but the data still contain related seasons, related teams, and shared league

conditions. The model should be read as a useful prediction tool for this dataset, not as a perfect causal model.

5 Discussion and Contribution

The main research question asked whether season-level NFL team wins can be predicted with a multi-source team-level dataset. The results suggest yes. The best model reached an R^2 of 0.907 and an MAE of 0.887 wins. This means the model explained much of the variation in the held-out seasons and usually missed by less than one win. The expected outcomes were mostly met. The project produced a model that predicts season wins and identified important features. The strongest features were mostly performance features, such as total yards, offensive WPA, first downs, defensive EPA per play, rushing attempts, and touchdowns. These findings fit basic football logic. Teams that gain yards, create first downs, score touchdowns, and limit opponent value tend to win more games.

The study also makes a contribution by combining several sources into one team-season dataset. The literature often studies one piece of the game at a time. This project uses a broader table that includes performance, roster, injuries, draft value, schedule, and past success. That design gives a wider view of team success. The most useful interpretation is not that LSTM is much better than all other models. The LSTM result was the best, but Ridge Regression was nearly equal. This suggests that strong feature engineering was more important than model complexity. In a small dataset, a simple model can compete with deep learning when the features are well built.

There are several limits. First, the dataset has only 320 team-season rows, which is a small sample for neural networks. Second, many predictors were measured during the same season as the target wins. This includes total yards, first downs, touchdowns, and EPA. Because of that, the model is best read as a season-level success model, not as a true preseason forecast. Third, some team labels changed across the years, so the dataset contains 35 team labels instead of 32 current franchises. Fourth, injury burden was measured with injury report counts, not missed snaps or injury severity. Fifth, draft quality was measured with a simple pick-value proxy, not with later player performance. The tuning setup is also a limit. The GRU and LSTM hyperparameter searches scored trials on the same test period used for final reporting. Future work should separate training, validation, and test seasons. For example, 2015 to 2020 could be used for training, 2021 could be used for validation, and 2022 to 2024 could be held out for final testing.

Future research should add point differential, preseason betting win totals, coaching changes, quarterback starts, opponent-adjusted efficiency, and player-level snap data. Future work should also build a true forecasting version that uses only information known before Week 1. That would make the model more useful for front offices, analysts, and fans before the season starts.

6 Conclusion

This study built a machine learning pipeline to predict NFL team wins at the season level. The final dataset joined public NFL data from 2015 to 2024 into 320 team-season observations. The study tested Linear Regression, Ridge Regression, Random Forest, XGBoost, MLP, GRU, and LSTM models. The best model was the Optuna-tuned LSTM, with an RMSE of 1.131, MAE of 0.887, and R^2 of 0.907. GRU and Ridge Regression were almost as strong. The feature results showed that total yards, offensive WPA, first downs, defensive EPA per play, and rushing attempts were the most important tree-based predictors.

The project fills a gap by focusing on season-level team success instead of only games, plays, or players. It also shows that a multi-source team-level dataset can predict season wins well. The main lesson is simple. Careful feature design mattered more than extra model complexity. Because many predictors came from the same season as the target, the model should be read as a season-level success model, not as a pure preseason forecast.

7 Team Contribution Statement

Brittany Fetterman contributed to the research background, literature review, project framing, coding and analysis, and review of the project findings.

Seth Phillips contributed to data collection, coding and analysis, model development, notebook implementation, and review of model results.

Ethan Stanks contributed to data preprocessing, coding and analysis, model testing, results review, and paper development.

All team members contributed to the overall project direction, interpretation of results, and completion of the research article.

Declarations

Funding. Not applicable.

Conflict of interest. The authors declare no conflict of interest.

Ethics approval. Not applicable. This project used public sports data and did not include human subject research.

Data availability. The dataset was built from public NFL data collected through the notebook workflow. The cleaned dataset generated in this study was saved as `nfl_master_dataset.csv`.

Code availability. The code for this project is available in the public repository at <https://github.com/EthanStanks/NFL-Team-Success>.

References

Brill, Ryan S., Yurko, Ronald, Wyner, Abraham J. (2025) Exploring the difficulty of estimating win probability: A simulation study. ArXiv:2406.16171 [stat.ME]

- da Silva, Gustavo Pompeu, de Andrade Moral, Rafael (2021) Frame by frame completion probability of an NFL pass. ArXiv:2109.08051 [stat.ML]
- Gifford, Matt, Bayrak, Tuncay (2023) A predictive analytics model for forecasting outcomes in the national football league games using decision tree and logistic regression. Decision Analytics Journal 8:100296. <https://doi.org/10.1016/j.dajour.2023.100296>
- Juuri, Sebastian (2023) Predicting the results of nfl games using machine learning. Master's thesis, Aalto University School of Business
- Mahoney, Joseph M., Paniak, Tomasz B. (2023) Method and validation for optimal lineup creation for daily fantasy football using machine learning and linear programming. ArXiv:2309.15253 [cs.LG]
- Obradović, Alden, Kečo, Dino (2024) Sports results prediction model using machine learning. SAR Journal 7(3):184–189. <https://doi.org/10.18421/SAR73-03>
- Skoki, Arian, Lerga, Jonatan, Stajduhar, Ivan (2022) ML-based approach for NFL defensive pass interference prediction using gps tracking data. ArXiv:2206.13222 [cs.LG]
- Szekely, Brian, Sinnott, Christian, Halow, Savannah, et al (2023) Nfl career success as predicted by nfl scouting combine. ArXiv:2303.05774 [cs.LG]
- Weirich, Caroline, Kim, Jun Woo, Yoon, Youngmin, et al (2025) Advancing NFL win prediction: from pythagorean formulas to machine learning algorithms. Frontiers in Sports and Active Living 7:1638446. <https://doi.org/10.3389/fspor.2025.1638446>